

Правила гарного Python-коду

Як писати так, щоб код розумів не лише комп'ютер, а й людина

Навіщо взагалі правила?

Комп'ютеру байдуже, який вигляд має твій код — він зрозуміє будь-який. Правила потрібні **людям**.

Уяви зошит з математики. Можна написати все в один рядок без пробілів — і ти сам за тиждень не розбереш, що там. А можна акуратно, по рядочках — і все зрозуміло з першого погляду.

Код — це зошит, який читають: ти через місяць, твій викладач, твоя команда. Офіційний збірник цих правил називається **PEP 8**.

Головна думка: код пишуть один раз, а читають десятки разів.

Частина 1. Як розставляти пробіли

1 Відступ — рівно 4 пробіли

Відступ у Python — це не краса, а **зміст**. Він показує, що міститься всередині умови або циклу. Зсунув рядок — змінив роботу програми.

✓ Правильно

```
if age >= 18:
    print("Можна")
    print("Усередині if")
```

✗ Неправильно

```
if age >= 18:
    print("Можна")
    print("Помилка!")
```

Чому: в інших мовах блоки позначають дужками {}, а в Python — лише відступами. Це фігурні дужки Python.

2 Пробіли навколо знаків =, +, >

Ліворуч і праворуч від знака — по одному пробілу. Як у звичайному письмі: ми пишемо «мама мила раму», а не «мамамилараму».

✓ Правильно

```
x = 10
total = price + tax
if x > 5:
```

✗ Неправильно

```
x=10
total=price+tax
if x>5:
```

3 Після коми — пробіл, перед дужкою — ні

✓ Правильно

```
numbers = [1, 2, 3]  
print("Привіт", name)
```

✗ Неправильно

```
numbers = [1,2,3]  
print ( "Привіт" , name )
```

Запам'ятай: пробіл ставлять *після* коми, а не перед — так само, як в українській мові.

4 Рядок — не довший за екран

Класичне правило — до 79 символів, сучасне — приблизно до 100. PyCharm малює вертикальну смужку-підказку: не заїжджай за неї.

Чому: довгий рядок доводиться прокручувати вбік, і кінець думки губиться. Короткі рядки читаються очима повністю.

5 Порожні рядки розділяють думки

Між функціями — **два** порожні рядки. Усередині функції, між змістовними шматочками — **один**.

```
def greet(name):  
    print("Привіт,", name)  
  
def farewell(name):  
    print("Бувай,", name)
```

Аналогія: це абзаци в книжці. Без них текст перетворюється на суцільну стіну літер.

Частина 2. Як називати речі

6 Ім'я має пояснювати, що всередині

Погане ім'я — це наче підписати всі банки на кухні літерою «б». Гарне ім'я позбавляє потреби щось пояснювати.

✓ Зрозуміло

```
speed = 120
user_name = "Ігор"
total_price = 250
```

✗ Загадка

```
s = 120
un = "Ігор"
tp = 250
```

Виняток: короткі `i`, `x`, `n` можна в маленьких циклах — там і так усе видно.

7 Слова з'єднуємо нижнім підкресленням

У Python імена пишуть маленькими літерами, а слова розділяють знаком «`_`». Такий стиль називають **змійним** (`snake_case`) — ім'я і справді схоже на змійку.

Що називаємо	Як пишемо	Приклад
Змінна	маленькими, через <code>_</code>	<code>user_age</code>
Функція	маленькими, через <code>_</code>	<code>get_name()</code>
Константа (не змінюється)	ВЕЛИКИМИ	<code>MAX_SPEED</code>
Клас	З Великих Літер Разом	<code>UserAccount</code>

8 Імена — лише англійськими літерами

✓ Правильно

```
age = 10
color = "red"
```

✗ Так не роблять

```
вік = 10
колір = "червоний"
```

Чому: українські імена Python дозволяє, але код перестають розуміти всі, крім тебе. А ще легко переплутати українську «с» з англійською «c» — вони виглядають однаково, а для комп'ютера це різні літери. Таку помилку шукають годинами.

Частина 3. Коментарі та дрібниці

9 Коментар пояснює «чому», а не «що»

✓ Корисно

```
# Знижка лише в будні
if day < 5:
    price = price * 0.9
```

✗ Марно

```
# множимо ціну на 0.9
price = price * 0.9
```

Чому: що робить рядок — і так видно з коду. А от *навіщо* — знаєш лише ти. Це й записуй.

10 Лапки — однакові в усьому файлі

Python однаково приймає 'кіт' і "кіт". Обери один вид і тримайся його — зазвичай подвійні.

11 Не порівнюй з True і None через ==

✓ По-пітонівськи

```
if is_ready:
    if name:
        if value is None:
```

✗ Задовго

```
if is_ready == True:
    if name != "":
        if value == None:
```

Чому: if is_ready: читається як англійська фраза «якщо готовий». Коротше — і помилитися ніде.

12 Імпорти — згори файлу, по одному в рядку

✓ Правильно

```
import os
import sys
```

✗ Неправильно

```
import os, sys
```

Шпаргалка: перевір себе перед задачею

- **Відступи** — скрізь по 4 пробіли, табуляція не перемішана з пробілами
- **Пробіли** — навколо =, +, > і після ком
- **Імена** — зрозумілі, англійські, маленькими_через_підкреслення
- **Рядки** — не довші за смужку-підказку в PyCharm
- **Порожні рядки** — два між функціями, один між шматочками думки
- **Коментарі** — там, де неочевидно «чому»
- **Хвилясті лінії** в PyCharm — виправлені, а не проігноровані

□ Секрет: PyCharm вміє зачісувати код сам

Натисни **Ctrl + Alt + L** — і PyCharm сам розставить пробіли та порожні рядки за правилами PEP 8.

Але не покладайся на це повністю: **імена змінних і коментарі** за тебе не вигадає ніхто. Найважливіше в цих правилах машина зробити не може.

Офіційний стандарт стилю Python називається PEP 8 · Гарні правила стають звичкою приблизно за місяць